

Git & Version Control Guide

What version control is, how Git and GitHub work, and exactly how this project's repository is organized, so the next team can pick it up safely.

Audience	Developers taking over the project, and US Tape IT staff who will own the code
Repository	github.com/Saul2525/US-Tape---Time-Tracking-Application
State as of	27 September 2026 (main at 5fe5965, ci-testing at 841bf65)
Authors	Gabriel Kagwanja, Chris Anderson, Saul Toribio
Companion docs	README.md, XAMPP Hosting Guide (PDF), ISSUES_AND_IMPROVEMENTS.md

Contents

1. Summary	3
2. What version control is	3
3. Git and GitHub	3
4. This project's repository	5
5. Branches	6
6. Project history	6
7. Case study: PR #2 and the revert	7
8. Automated tests and CI	7
9. Day-to-day workflow	8
10. Handoff checklist	10
11. Command cheat sheet	11

1. Summary

All source code for the Time Tracking Application is kept in a **Git repository** hosted on **GitHub**. Git records every change ever made: who made it, when, and why. Any past version can be restored, and several people can work at once without overwriting each other.

This guide covers the concepts first (Sections 2–3), then this repository specifically: its branches, history and automated tests (Sections 4–8). It ends with the day-to-day workflow and a **handoff checklist** (Sections 9–10).

The three things that matter most at handoff

- **Ownership:** the repository lives on a student's personal GitHub account (Saul2525). Transfer it to a US Tape-owned account or organization before the team leaves (Section 10).
- **Tests aren't on main yet:** the automated tests and CI pipeline live on the `ci-testing` branch and still need to be merged (Section 8).
- **Delete** `revert-2-main`: it is a leftover branch. Merging it would delete the Excel export feature (Section 7).

2. What version control is

Version control is a system that records changes to a set of files over time. Instead of copies named `admin_final.php`, `admin_final_v2.php` and `admin_REAL_final.php`, there is one set of files plus a complete, searchable history of every version.

Problem without version control	What version control gives you
"The app worked last week. What changed?"	Every change is a dated, labelled snapshot. You can compare any two versions line by line
Two people edit the same file and one person's work is lost	Each person works separately; Git merges the changes and flags true conflicts instead of silently overwriting
A bad change breaks production	Undo it in one command, even months later
"Who wrote this and why?"	Each line can be traced to the commit and author that added it (<code>git blame</code>)
The laptop with the code dies	Every clone is a full backup of the whole history
A new developer joins	They clone the repository and get everything, history included

3. Git and GitHub

Git is the version-control program. It is free, runs on every operating system, and is the industry standard. It works entirely on your own computer: each copy of a repository holds the full history, so you can commit and browse history offline.

GitHub is a website that hosts Git repositories so a team can share one central copy. On top of Git it adds **Pull Requests** (proposing and reviewing changes before they join the main code), access control, issue tracking and **GitHub Actions** (automated testing). Git is the tool; GitHub is where the team's

shared copy lives.

3.1 How a change travels

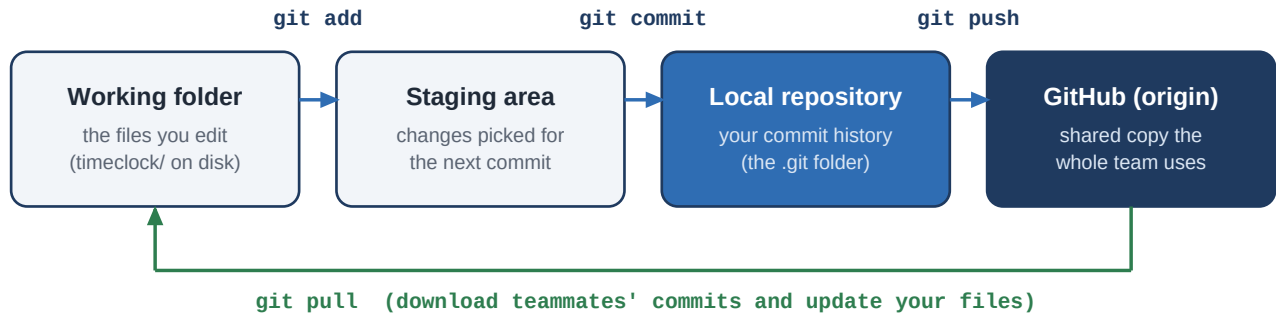


Figure 1: The four places a change passes through.

3.2 Key terms

Term	Meaning	In this project
Repository (repo)	A project folder tracked by Git, including its history (the hidden .git folder)	The timeclock/ folder
Commit	A saved snapshot of changes with an author, date and message	e.g. 5fe5965 "Changes to excel export"
Hash	A commit's unique ID. The first 7 characters are enough to refer to it	5fe5965
Branch	An independent line of work. Commits on one branch don't affect others until merged	main, ci-testing
Merge	Combines one branch's commits into another	PR #1 and PR #2
Remote / origin	The shared copy on GitHub. origin is its default nickname	github.com/Saul2525/...
Clone / pull / push	Download the repo once / fetch new commits / upload your commits	
Pull Request (PR)	A GitHub page proposing to merge a branch, with review and CI checks	PR #1, PR #2
.gitignore	List of files Git must never track	Excludes config.php (passwords)
CI	Continuous Integration: tests that run automatically on each push or PR	GitHub Actions on ci-testing

4. This project's repository

Item	Value
GitHub URL	<code>https://github.com/Saul2525/US-Tape---Time-Tracking-Application</code>
Clone (SSH)	<code>git@github.com:Saul2525/US-Tape---Time-Tracking-Application.git</code>
Owner	Saul Toribio's personal GitHub account (must be transferred , Section 10)
Default branch	<code>main</code> : the code that should be deployed
Repository root	The <code>timeclock/</code> folder. The parent folder <code>US Tape Time Tracking Application/</code> is not a repository, so run all Git commands from inside <code>timeclock/</code> . The <code>ERD Diagram.png</code> in the parent folder is an untracked copy
Tracked on main	All PHP pages, <code>db.sql</code> , <code>README.md</code> , <code>ISSUES_AND_IMPROVEMENTS.md</code> , <code>TEMP ERD Diagrams.png</code> , <code>.gitignore</code> , and <code>docs/</code> (this handoff)
Extra on ci-testing	<code>.github/workflows/ci.yml</code> , <code>composer.json</code> , <code>composer.lock</code> , <code>phpunit.xml</code> , <code>tests/</code> , <code>test_helpers/</code>

4.1 What is deliberately NOT in Git

.gitignore entry	Why
<code>config.php</code>	Holds database credentials. Each machine creates its own (template in the <code>README</code>). The history was checked and <code>config.php</code> has never been committed. Keep it that way
<code>/vendor/</code>	Composer-downloaded packages (PHPUnit). Rebuilt with <code>composer install</code>
<code>.DS_Store</code>	macOS Finder clutter
<code>.phpunit.result.cache</code> (ci-testing)	Test runner cache

4.2 Contributors

Person	Git identities seen in history	Main contributions
Gabriel Kagwanja	<code>gak227</code> , <code>Gabrielkag</code>	Initial system, post-IT rewrite (PIN clock, manager login, shift editing, audit log), <code>README</code> , handoff docs
Chris Anderson	Chris Anderson, <code>cha727</code>	Initial upload, admin employee management, time UI, Excel (CSV) export, issues list
Saul Toribio	Saul (two emails)	Repository owner; PHPUnit tests, GitHub Actions CI, merged PRs #1 and #2

Several people committed under more than one name or email. Before your first commit on a new machine, set one consistent identity (Section 11).

5. Branches

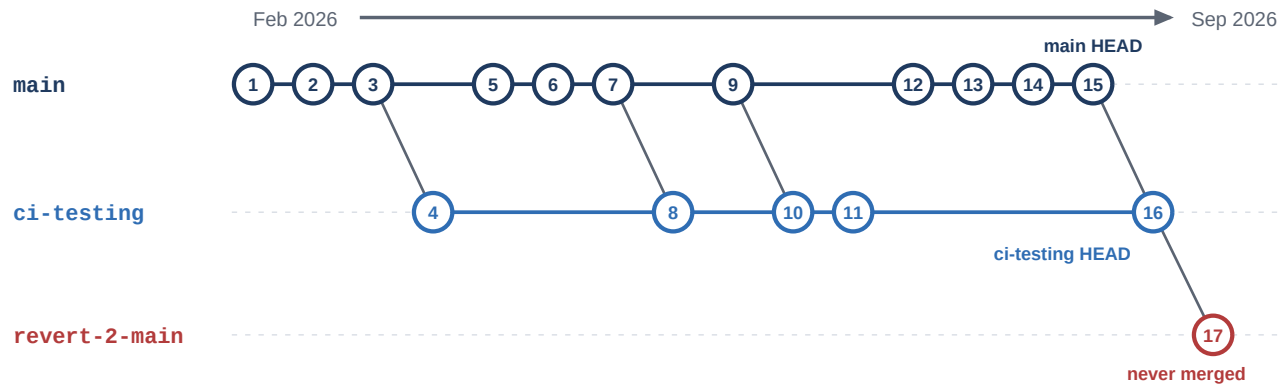


Figure 2: The real commit graph, simplified. Numbers match the table in Section 6. Grey lines are merges between branches.

Branch	Purpose	Status (27 Sep 2026)	Recommended action
main	The deployable application	Up to date with GitHub at 5fe5965. Contains every feature	Keep as the only deploy source. Protect it (Section 10)
ci-testing	Saul's automated tests + GitHub Actions	Contains all of main plus the test files, and nothing else differs	Open a PR ci-testing → main so tests reach main (Section 8)
revert-2-main	Auto-created by GitHub's "Revert" button on PR #2	One commit that <i>removes</i> the export feature and issues list. Never merged	Delete it. Never merge it

6. Project history

All dates are 2026. Messages are paraphrased. Run `git log --oneline --graph --all` to see the originals.

#	Commit	Date	By	What happened
1	7363703	Feb 16	Gabriel	Initial timeclock system commit
2	d463c47	Feb 16	Gabriel	"Polishing": merged Chris's first upload (a0686c3, Feb 12)
3	885d013	Feb 16	Gabriel	README added through the GitHub website
4	0c6b7dc	Mar 23	Saul	ci-testing starts: PHPUnit tests + CI workflow
5	2a41ead	Apr 14	Gabriel	Rewrite after US Tape IT confirmation: PIN-only clock, manager login, dashboard, shift editing, audit log
6	666b78b	Apr 14	Gabriel	Merged the website README (3) into the local work
7	73b0c81	Apr 20	Chris	Admin employee management (admin.php), DB population
8	6ab6002	Apr 21	Saul	PR #1: brought main into ci-testing

#	Commit	Date	By	What happened
9	4e80981	Apr 21	Chris	Time-entry UI updates, Export button (placeholder)
10	11604bf	Aug 29	Saul	Merged main into ci-testing again
11	1fb3769...05a7207	Aug 29–Sep 1	Saul	4 commits: tests run against real MySQL via cURL; ClockHandlerTest + CreateUserTest; readiness checks
12	993474b	Aug 31	Chris	ISSUES_AND_IMPROVEMENTS.md ("To Do")
13	fb1f7ed	Sep 1	Gabriel	README updated with setup instructions
14	427ee4c	Sep 1	Gabriel	Merged Chris's issues list with the README work
15	5fe5965	Sep 1	Chris	Working Excel (CSV) export : current tip of main
16	841bf65	Sep 3	Saul	PR #2: brought main into ci-testing: current tip of ci-testing
17	274390e	Sep 3	Saul	Revert of PR #2 on branch revert-2-main (never merged)

7. Case study: PR #2 and the revert

This short episode is worth understanding, because it's the one place in the repository where the wrong click could lose work.

1. On Sep 3, PR #2 merged the latest main (including the new export feature) **into** ci-testing, so the tests would run against current code.
2. Minutes later GitHub's **Revert** button was used on PR #2. That button doesn't change anything by itself. It creates a new branch (revert-2-main) with a commit that undoes the PR, and offers a new PR to merge it.
3. That revert PR was never merged. So ci-testing still holds the full, current code, and revert-2-main is an orphan branch whose only change deletes `export_shifts.php`, the issues list and part of the README.

What to do

Delete revert-2-main on GitHub (Branches page → trash icon) or run `git push origin --delete revert-2-main`. Nothing is lost: the commit it reverts is still in main and ci-testing.

Lesson: on GitHub, "Revert", "Merge" and "Delete branch" buttons act on the shared copy everyone uses. Read the PR's *Files changed* tab before merging anything.

8. Automated tests and CI

Continuous Integration (CI) means a server automatically builds and tests the code every time it changes, so problems are caught before they reach main. This project uses **GitHub Actions**, configured in `.github/workflows/ci.yml` on the ci-testing branch.

8.1 When it runs

- On every push to `ci-testing`.
- On every Pull Request into `main`, but only if the PR's branch contains the workflow file. Right now that means PRs from `ci-testing`.

8.2 What it does

1. Starts a fresh Ubuntu machine with a MySQL 8.0 database.
2. Installs PHP 8.5 and PHPUnit 13 (`composer install`).
3. Imports `db.sql` and writes a `CI-only config.php` from environment variables.
4. Starts `php -S 127.0.0.1:8000` and waits until it responds.
5. Runs `composer test`. The tests send real HTTP requests with `cURL` to `clock_handler.php` and `create_user.php` and check the database afterwards.
6. Marks the commit or PR with a green check or red cross on GitHub.

8.3 Known gaps

- **Not on main yet.** Merge it with a PR from `ci-testing` → `main`. The CI will run on that PR itself, which is a good first check.
- **Tests target** `create_user.php`, a legacy page the security review says to delete. Before deleting it, move those tests to `admin.php`, which needs a logged-in admin session in the test.
- `db.sql` **drift**: CI builds its database from `db.sql`, which lacks the `password` column. Manager-login tests will fail until `db.sql` is updated (see README).
- No tests yet for manager pages, shift edits, export or admin.

9. Day-to-day workflow

Use the **feature-branch workflow**: never commit directly to `main`. Each change gets its own short-lived branch, a Pull Request, and a green CI run before merging.

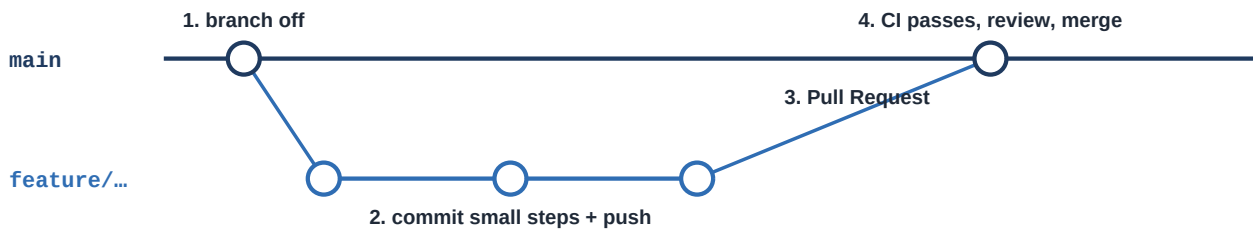


Figure 3: Feature-branch workflow.

9.1 First-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@ustape.com" # same email as your GitHub account
git clone git@github.com:Saul2525/US-Tape---Time-Tracking-Application.git timeclock
cd timeclock
# create config.php (template in README.md), then follow the XAMPP guide
```


9.2 Every change

```
git switch main
git pull                                # start from the latest shared code
git switch -c fix/hash-manager-passwords # 1. new branch, named after the task

# ...edit files, test locally in the browser...

git status                             # see what changed
git diff                               # review the actual lines
git add manager_login.php admin.php    # stage specific files (avoid "git add ." blindly)
git commit -m "fix: hash manager passwords with password_hash()"
git push -u origin fix/hash-manager-passwords
```

1. On GitHub, click **Compare & pull request**. Base: main.
2. Describe *what* changed, *why*, and *how you tested it*. Link the related item in ISSUES_AND_IMPROVEMENTS.md and tick it off in the same PR.
3. Wait for CI to go green; ask a teammate to review the *Files changed* tab.
4. Merge, then click **Delete branch**. Locally: `git switch main && git pull`.
5. If the change needs a database change, include the SQL in the PR and in `db.sql`, so the schema never drifts again.

9.3 Commit messages

Saul's commits already follow the **Conventional Commits** style. Please use it for everything: a type prefix plus a short, specific summary.

Prefix	Use for	Good example
feat:	New feature	feat: add approval note field to shift editor
fix:	Bug fix	fix: reject clock-out earlier than clock-in in update_shift.php
refactor:	Code change with no behaviour change	refactor: move role check into auth.php
docs:	Documentation only	docs: add XAMPP and Git handoff guides
test:	Tests only	test: cover admin.php employee creation
chore:	Tooling, config, dependencies	chore: bump PHPUnit to 13.1

Avoid messages like "changes" or "More changes" (both appear in this history). Six months later nobody can tell what they did.

10. Handoff checklist

Task	How
[] Transfer repository ownership to a US Tape GitHub organization or IT-owned account	Owner (Saul): Settings → General → Danger Zone → <i>Transfer ownership</i> . GitHub redirects the old URL, but everyone should update remotes: <code>git remote set-url origin <new-url></code>
[] Give the next maintainers access; remove departing members' write access afterwards	Settings → Collaborators and teams
[] Commit the handoff docs (README.md, ISSUES update, docs/*.pdf)	docs : commit via a PR into main
[] Merge ci-testing into main	PR ci-testing → main; CI runs on it; merge when green
[] Delete revert-2-main	Section 7
[] Protect main	Settings → Branches → rule for main: require a PR, require the "PHP CI / test" check, block force-pushes
[] Tag the handoff version	<code>git tag -a v1.0-handoff -m "Handoff Sept 2026"</code> <code>&& git push origin v1.0-handoff</code>
[] Confirm no secrets in history	Verified 27 Sep 2026: config.php never committed. Note: the three sample kiosk PINs appear in README history (commit 5fe5965). They are dev-only and must never be reused for real employees
[] Hand over the dev database if needed	It isn't in Git. <code>mysqldump -u root timeclock > timeclock_dev.sql</code> and share it privately, not via GitHub

11. Command cheat sheet

Goal	Command
Get the project the first time	<code>git clone <url> timeclock</code>
See what's changed	<code>git status</code> · <code>git diff</code>
Get teammates' latest work	<code>git pull</code>
Start a new piece of work	<code>git switch -c feat/short-name</code>
Save a snapshot	<code>git add <files></code> then <code>git commit -m "type: message"</code>
Upload your branch	<code>git push -u origin <branch></code>
List branches (local + GitHub)	<code>git branch -a</code>
View history as a graph	<code>git log --oneline --graph --all</code>
Who last changed each line	<code>git blame admin.php</code>
See one old commit	<code>git show 5fe5965</code>
Throw away uncommitted edits to a file	<code>git restore <file></code>
Unstage a file (keep the edits)	<code>git restore --staged <file></code>
Undo last commit, <i>not yet pushed</i>	<code>git reset --soft HEAD~1</code>
Undo a commit <i>already pushed</i>	<code>git revert <hash></code> (adds a new "undo" commit)
Park unfinished work temporarily	<code>git stash</code> · <code>git stash pop</code>

11.1 Common problems

Situation	What to do
Merge conflict after pull or merge	Git marks the file with <<<<<<, =====, >>>>>>. Edit it to the correct final version, remove the markers, then <code>git add <file></code> and <code>git commit</code> . VS Code shows Accept Current / Incoming buttons for this
Push rejected ("fetch first")	Someone pushed before you. <code>git pull</code> , fix any conflicts, push again. Never use <code>--force</code> on main
Committed a password or secret	Change the password first. It is now exposed. Then remove it from history (GitHub docs: "Removing sensitive data") and tell the owner
Committed to main by mistake (not pushed)	<code>git switch -c my-branch</code> (keeps the commit), then <code>git switch main</code> && <code>git reset --hard origin/main</code>
Git commands say "not a git repository"	You're in the parent folder. <code>cd timeclock</code>
"Permission denied (publickey)"	Your SSH key isn't on GitHub. Add it under GitHub → Settings → SSH keys, or clone with the HTTPS URL instead